

Vulkan Notes

Instance, Validation, Devices & Queues

Adam HAMOUCH

June 2026

1. Vulkan — Vue d'ensemble

API graphique cross-platform, plus bas niveau qu'OpenGL. Adapte aux GPU modernes : meilleur controle, meilleures performances, mais plus verbeux. En Vulkan on remplit des structs plutot que passer des parametres a des fonctions, et on informe toujours le driver du type envoye via `sType`. Le champ `pNext` permet de chainer des extensions sur chaque struct.

Etapas pour dessiner un triangle :

1. **Instance + Physical Device** : configurer l'API et le hardware.
2. **Logical Device + Queue Families** : choisir les features, configurer les queues.
3. **Window Surface + Swap Chain** : fenetrage et cibles de rendu.
4. **Image Views + Framebuffers** : vues sur les images du swap chain.
5. **Render Pass** : decrit les images utilisees et comment les traiter.
6. **Graphics Pipeline** : shaders, rasterisation, blend, depth.
7. **Command Pools + Command Buffers** : enregistrer les commandes de rendu.
8. **Boucle principale** : acquérir image, soumettre commandes, presenter.

2. Instance

Connexion entre ton application et le driver Vulkan. Point d'entree de l'API — avant l'instance, Vulkan n'existe pas pour ton programme. Definit la version Vulkan, les extensions globales et les validation layers.

```
VkApplicationInfo appInfo{};
appInfo.sType      = VK_STRUCTURE_TYPE_APPLICATION_INFO;
appInfo.pApplicationName = "Mon_Moteur";
appInfo.apiVersion  = VK_API_VERSION_1_4;

VkInstanceCreateInfo createInfo{};
createInfo.sType      = VK_STRUCTURE_TYPE_INSTANCE_CREATE_INFO;
createInfo.pApplicationInfo = &appInfo;
createInfo.enabledExtensionCount = extensions.size();
createInfo.ppEnabledExtensionNames = extensions.data();
createInfo.enabledLayerCount = layers.size();
createInfo.ppEnabledLayerNames = layers.data();

VkInstance instance;
vkCreateInstance(&createInfo, nullptr, &instance);

// Enumerer les extensions disponibles
vkEnumerateInstanceExtensionProperties(nullptr, &extCount, extensions.data());

// L'instance doit etre detruite a la fin du programme
vkDestroyInstance(instance, nullptr);
```

3. Validation Layers

Vulkan n'embarque pas d'error handling pour les performances. Les validation layers sont des surcouches fournies par le SDK qui verifient les valeurs passees, la gestion memoire et la securite des threads. A activer en **debug uniquement** — elles divisent le framerate par 2-3.

```
const std::vector<const char*> validationLayers = {
    "VK_LAYER_KHRONOS_validation"
};

#ifdef NDEBUG
    constexpr bool enableValidationLayers = false; // release : desactivees
#else
    constexpr bool enableValidationLayers = true;  // debug : actives
#endif

// Callback pour capter les messages de debug proprement
VkDebugUtilsMessengerCreateInfoEXT debugInfo{};
debugInfo.sType = VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT;
debugInfo.messageSeverity = VK_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT
    | VK_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT;
debugInfo.messageType = VK_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT;
debugInfo.pfnUserCallback = debugCallback;
```

Regle pratique : si le programme crash ou affiche des artefacts, la premiere chose a verifier ce sont les messages des validation layers.

4. Physical Device et Queue Families

On enumere les GPU disponibles et on verifie lequel convient. Le physical device n'est pas cree — il est **decouvert**. Sa destruction est liee au cycle de vie de l'instance, pas besoin de le detruire manuellement.

```
// Enumerer les GPU
uint32_t deviceCount = 0;
vkEnumeratePhysicalDevices(instance, &deviceCount, nullptr);
if (deviceCount == 0) throw std::runtime_error("aucun GPU Vulkan!");
std::vector<VkPhysicalDevice> devices(deviceCount);
vkEnumeratePhysicalDevices(instance, &deviceCount, devices.data());

// Verifier les capacites de chaque GPU
bool isDeviceSuitable(VkPhysicalDevice device) {
    VkPhysicalDeviceProperties props; // nom, type, limites
    vkGetPhysicalDeviceProperties(device, &props);
    VkPhysicalDeviceFeatures features; // ce que le GPU sait faire
    vkGetPhysicalDeviceFeatures(device, &features);
    QueueFamilyIndices indices = findQueueFamilies(device);
    return props.deviceType == VK_PHYSICAL_DEVICE_TYPE_DISCRETE_GPU
        && indices.isComplete();
}
```

Queue Families

Le GPU organise ses files d'attente en familles (graphics, compute, transfer...). On cherche l'index de chaque famille dont on a besoin. `std::optional` permet de distinguer "trouve a l'index 0" de "pas trouve".

```
struct QueueFamilyIndices {
    std::optional<uint32_t> graphicsFamily;
    std::optional<uint32_t> presentFamily;
    bool isComplete() {
```

```

        return graphicsFamily.has_value() && presentFamily.has_value();
    }
};

QueueFamilyIndices findQueueFamilies(VkPhysicalDevice device) {
    QueueFamilyIndices indices;
    uint32_t count = 0;
    vkGetPhysicalDeviceQueueFamilyProperties(device, &count, nullptr);
    std::vector<VkQueueFamilyProperties> families(count);
    vkGetPhysicalDeviceQueueFamilyProperties(device, &count, families.data());
    for (uint32_t i = 0; i < families.size(); i++) {
        if (families[i].queueFlags & VK_QUEUE_GRAPHICS_BIT)
            indices.graphicsFamily = i;
        // present check via vkGetPhysicalDeviceSurfaceSupportKHR
    }
    return indices; // findQueueFamilies decouvre, isDeviceSuitable decide
}

```

5. Logical Device et Queues

Canal de communication avec le GPU. Cree a partir d'une struct qui prend les infos du physical device : queues a preparer, features GPU, extensions, validation layers. Doit etre detruit avant l'instance.

```

QueueFamilyIndices indices = findQueueFamilies(physicalDevice);

// Preparer les queues a creer
std::set<uint32_t> uniqueFamilies = {
    indices.graphicsFamily.value(), indices.presentFamily.value()
};
std::vector<VkDeviceQueueCreateInfo> queueCreateInfos;
float priority = 1.0f;
for (uint32_t family : uniqueFamilies) {
    VkDeviceQueueCreateInfo queueInfo{};
    queueInfo.sType = VK_STRUCTURE_TYPE_DEVICE_QUEUE_CREATE_INFO;
    queueInfo.queueFamilyIndex = family;
    queueInfo.queueCount = 1;
    queueInfo.pQueuePriorities = &priority;
    queueCreateInfos.push_back(queueInfo);
}

VkPhysicalDeviceFeatures deviceFeatures{}; // tout VK_FALSE pour l'instant

VkDeviceCreateInfo createInfo{};
createInfo.sType = VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO;
createInfo.pQueueCreateInfos = queueCreateInfos.data();
createInfo.queueCreateInfoCount = queueCreateInfos.size();
createInfo.pEnabledFeatures = &deviceFeatures;
createInfo.enabledExtensionCount = deviceExtensions.size();
createInfo.ppEnabledExtensionNames = deviceExtensions.data();

VkDevice device;
vkCreateDevice(physicalDevice, &createInfo, nullptr, &device);

// Recuperer les references aux queues creees
VkQueue graphicsQueue, presentQueue;
vkGetDeviceQueue(device, indices.graphicsFamily.value(), 0, &graphicsQueue);
vkGetDeviceQueue(device, indices.presentFamily.value(), 0, &presentQueue);

// Destruction
vkDestroyDevice(device, nullptr); // avant vkDestroyInstance

```

Ordre de destruction : toujours detruire dans l'ordre inverse de creation. Queues detruites avec le device, device avant l'instance. Sinon comportement indefini.